Unit II :: Arrays, Vectors and Matrix Operations

SEC-01-PHY-91-R-03

Dr. Parag Bhattacharya
Department of Physics
Rangapara College (Autonomous)

Syllabus

Unit II: Arrays, Vectors and Matrix Operations

Creation of row and column vectors, matrices; special matrices: zeros, ones, identity; indexing and slicing: accessing elements, sub-matrices; concatenation and reshaping; operations: element-wise vs matrix multiplication; matrix computations: transpose, inverse, determinant, eigenvalues; solving linear systems.

1 Arrays, matrices and vectors

- *Array*: A collection of elements arranged in a regular grid with one or more dimensions. Elements of a Scilab numeric array share the same numeric type. Arrays may be 1-D, 2-D, or higher dimensional.
- *Matrix*: A two-dimensional numeric array with rows and columns. Matrices are the primary data structure for linear algebra in Scilab.
- *Vector*: A one-dimensional array. In Scilab a vector is represented either as a row vector (1 × n) or a column vector (n × 1); both are special cases of matrices.

2 Creation of Scilab arrays

Scilab uses a concise, readable syntax for building numeric arrays. A literal array is written between square brackets `[...]`: spaces or commas separate entries in the same row, and a semicolon `(;)` separates rows. Thus `[1 2 3]` is a 1×3 row vector while `[1; 2; 3]` is a 3×1 column vector.

A 2×2 matrix is written as `[a b; c d]`. Whitespace and line breaks may be used to improve readability, but the bracketed structure determines the array shape.

```
--> r = [1  2  3]
r = [1x3 double]
1.   2.   3.

--> c = [1;  2;  3]
c = [3x1 double]
1.
2.
3.

--> m = [1  2; 3  4]
m = [2x2 double]
1.   2.
3.   4.

--> X = [1,2,5; 6,8,9; -1,6,4]
X = [3x3 double]
1.   2.   5.
6.   8.   9.
-1.   6.   4.
```

2.1 Special matrices

For common array initialisations Scilab provides utility functions that are convenient and efficient: `zeros(m,n)` and `ones(m,n)` create $m \times n$ arrays filled with zeros or ones. The function `eye(n)` builds the $n \times n$ identity matrix. Another Scilab function called `linspace(a,b,n)` is useful for creating evenly spaced vectors. These functions are preferable to typing large arrays by hand and make code easier to read and maintain.

```
--> zeros(2, 3)
ans = [2x3 double]
0.   0.   0.
0.   0.   0.

--> ones(3, 4)
ans = [3x4 double]
1.   1.   1.   1.
1.   1.   1.   1.
1.   1.   1.   1.

--> eye(5,5)
ans = [5x5 double]
1.   0.   0.   0.   0.
0.   1.   0.   0.   0.
0.   0.   1.   0.   0.
0.   0.   0.   1.   0.
0.   0.   0.   0.   1.

--> linspace(0, 1, 5)
ans = [1x5 double]
0.   0.25   0.5   0.75   1.
```

3 Working with arrays

3.1 Indexing and slicing

Indexing and slicing are the primary tools for accessing and manipulating parts of arrays. Scilab uses 1-based indexing and column-major storage; parentheses `()` are used for indexing.

3.1.1 Basic indexing rules

1. Indices start at 1.
2. Use `A(i,j)` to access the element in the ith row and j^{th} column.
3. A single index `A(k)` performs linear (column-wise) indexing into A (elements are counted down columns).
4. Attempting to index outside the array bounds raises an error – check using the `size()` function first.

```
--> A = [1 6 9 2 -3];

--> A(2)
ans =
6.

--> A(1)
ans =
1.

--> A(7)
Invalid index.
```

```
--> size(A)
ans = [1x2 double]
1.   5.

--> B = [4 7 9; 8 2 5; -6 3 1]
B = [3x3 double]
4.   7.   9.
8.   2.   5.
-6.   3.   1.

--> B(1,3)
ans =
9.
```

```
--> B(3,2)
ans =
3.

--> B(4,5)
Invalid index.

--> size(B)
ans = [1x2 double]
3.   3.
```

3.1.2 Basic slicing rules

1. Use `:` to select an entire row or column.
2. Use `start:end` to select a range of indices.
3. Use `$` to refer to the last index.

```
--> A = [1 2 -2 3; 4 -4 7 9; 0 -1 5 8]
A = [3x4 double]
1.   2.  -2.   3.
4.  -4.   7.   9.
0.  -1.   5.   8.

--> A(2,:)
ans = [1x4 double]
4.  -4.   7.   9.

--> A(:,3)
ans = [3x1 double]
-2.
7.
5.
```

```
--> A(2:3,2:4)
ans = [2x3 double]
-4.   7.   9.
-1.   5.   8.

--> A(:,2:3)
ans = [3x2 double]
2.  -2.
-4.   7.
-1.   5.

--> A($,$)
ans =
8.

--> A($,:)
ans = [1x4 double]
0.  -1.   5.   8.

--> A(:,$)
ans = [3x1 double]
3.
9.
8.
```

3.2 Reshaping arrays

Reshaping an array means changing its dimensions (number of rows and columns) without altering the data it contains. This is often useful when one needs to reinterpret data, prepare it for mathematical operations, or display it in a specific format. Scilab provides simple and efficient tools for reshaping arrays.

The most common way to reshape an array in Scilab is by using the `matrix()` function. Its general syntax is: `B = matrix(A, m, n)`, where `A` is the original array and `m`, `n` are the required number of rows and columns.

```
--> A = [1 2 3 4 5 6 7 8 9]
A = [1x9 double]
1.  2.  3.  4.  5.  6.  7.  8.  9.

--> matrix(A, 3, 3)
ans = [3x3 double]
1.   4.   7.
2.   5.   8.
3.   6.   9.

--> B = matrix(A, 3, 3)
B = [3x3 double]
1.   4.   7.
2.   5.   8.
3.   6.   9.

--> C = matrix(A, 1, 9)
C = [1x9 double]
1.  2.  3.  4.  5.  6.  7.  8.  9.
```

The total number of elements must remain unchanged after reshaping, i.e. if A has $p \times q$ elements, then $m \times n$ must equal $p \times q$. Attempting to reshape into an incompatible size will produce an error.

3.3 Concatenating arrays

Array concatenation refers to the process of joining two or more arrays to form a larger array. In Scilab, this can be done either horizontally (side by side) or vertically (one below another) using simple bracket notation. Concatenation is widely used for building matrices from smaller blocks, combining results, or assembling data from different sources.

Scilab allows arrays to be concatenated using square brackets `[ ]`.

- Elements separated by spaces or commas (`,`) within brackets are placed horizontally (same row).
- Elements or arrays separated by semicolons (`;`) are placed vertically (new rows).

```
--> A = [1 2 3]
A = [1x3 double]
1.   2.   3.

--> B = [4 5 6]
B = [1x3 double]
4.   5.   6.

--> C = [A; B]
C = [2x3 double]
1.   2.   3.
4.   5.   6.

--> D = [A, B]
D = [1x6 double]
1.   2.   3.   4.   5.   6.
```

3.4 Size and length of arrays

The `size()` function returns the size of the array along all the dimensions. If it is a vector, then it returns the number of elements; in case of a matrix, it returns the number rows and columns.

The `length()` function returns the total number of elements in an array, regardless of whether it is a vector or a matrix.

```
--> A = [2  1; 5  7; 9  3]
A = [3x2 double]
2.   1.
5.   7.
9.   3.

--> size(A)
ans = [1x2 double]
3.   2.

--> length(A)
ans =
6.

--> B = [-4  8  9 -3;  1  7  2  -5]
B = [2x4 double]
-4.   8.   9.  -3.
1.   7.   2.  -5.

--> size(B)
ans = [1x2 double]
2.   4.

--> length(B)
ans =
8.
```

4 Operations on arrays

4.1 Element-wise vs. matrix multiplication

Matrix multiplication uses * and follows the mandatory dimension compatibility rule: if A is an $m \times n$ matrix and B is an $n \times p$ matrix, then `C = A * B` produces an $m \times p$ matrix.

```
--> A = [2  1; 5  7; 9  3]
A = [3x2 double]
2.   1.
5.   7.
9.   3.

--> size(A)
ans = [1x2 double]
3.   2.

--> B = [-4  8  9;  1 7  2]
B = [2x3 double]
-4.   8.   9.
1.   7.   2.
```

```
--> size(B)
ans = [1x2 double]
2.   3.

--> C = A * B
C = [3x3 double]
-7.    23.   20.
-13.   89.   59.
-33.   93.   87.

--> size(C)
ans = [1x2 double]
3.   3.
```

Element-wise multiplication operates on corresponding entries and uses the `.*` operator. Both operands must have the same shape.

```
--> A = [2  1; 5  7; 9  3]
A = [3x2 double]
2.   1.
5.   7.
9.   3.

--> B = [-4  8;  9  1;  7  2]
B = [3x2 double]
-4.   8.
9.   1.
7.   2.

--> C = A .* B
C = [3x2 double]
-8.    8.
45.   7.
63.   6.

--> D = [-4  8  9;  1  7  2]
D = [2x3 double]
-4.   8.   9.
1.   7.   2.

--> A .* D
Operator .*: Wrong dimensions for
 operation
[3x2] .* [2x3], same dimensions
 expected.
```

4.2 Matrix computations

4.2.1 Transpose

The transpose of a matrix is another matrix whose rows and columns are swapped. In Scilab, the transpose of a matrix `A` is obtained by the `'` operator, i.e. `A'` is conjugate transpose of `A`.

```
--> A = [2  1; 5  7; 9  3]
A = [3x2 double]
2.   1.
5.   7.
9.   3.

--> A'
ans = [2x3 double]
2.   5.   9.
1.   7.   3.

--> B = [-4  8  9  1  7]
B = [1x5 double]
-4.   8.   9.   1.   7.

--> B'
ans = [5x1 double]
-4.
8.
9.
1.
7.
```

4.2.2 Determinant

The determinant of a matrix is a single scalar value computed from a square matrix. In Scilab, the determinant of a square matrix `A` is obtained by the `det()` function.

```
--> A = [3  4  6; 7  -1  2; 8  5  9]
A = [3x3 double]
3.   4.   6.
7.  -1.   2.
8.   5.   9.

--> size(A)
ans = [1x2 double]
3.   3.

--> det(A)
ans =
13.000000
```

```
--> B = [2  1; 5  7; 9  3]
B = [3x2 double]
2.   1.
5.   7.
9.   3.

--> size(B)
ans = [1x2 double]
3.   2.

--> det(B)
det: Wrong type for input argument #1:
Square matrix expected.
```

4.2.3 Inverse

The inverse of a square matrix exists only if it is non-singular, i.e. its determinant exists. In Scilab, the matrix inverse is computed with the `inv()` function for a square, non-singular matrix.

```
--> A = [3 4 6; 7 -1 2; 8 5 9]
A = [3x3 double]
3.   4.   6.
7.  -1.   2.
```

```
 8.   5.   9.

--> det(A)
ans =
13.000000

--> inv(A)
ans = [3x3 double]
-1.4615385  -0.4615385   1.0769231
-3.6153846  -1.6153846   2.7692308
3.3076923   1.3076923  -2.3846154
```

```
--> B = [1  2; 2  4]
B = [2x2 double]
1.   2.
2.   4.

--> det(B)
ans =
0.

--> inv(B)
inv: Problem is singular.
```

4.2.4 Eigenvalues: solving linear systems

For a square matrix $\boldsymbol{A}$, a non-zero vector $\boldsymbol{v}$ is called an eigenvector and the number λ is called the corresponding if they satisfy the following matrix equation

$$\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v}$$

Geometrically, an eigenvector points in a direction that the matrix does not rotate, i.e. it is only stretched or shrunk by the factor λ.

Eigenvalues and eigenvectors are useful for finding natural modes, studying stability, and simplifying matrix problems; they are usually computed with software and checked by confirming that $\boldsymbol{A}\boldsymbol{v}$ is approximately equal to $\lambda\boldsymbol{v}$.

In Scilab, the eigenvectors and eigenvalues are obtained using the `spec()` function. This function returns two matrices – one containing the eigenvectors as columns, and the other containing the eigenvalues as diagonals.

```
--> A = [2 1; 1 2]
A = [2x2 double]
2.   1.
1.   2.

--> [V, D] = spec(A);

--> V
V = [2x2 double]
-0.7071068   0.7071068
0.7071068   0.7071068

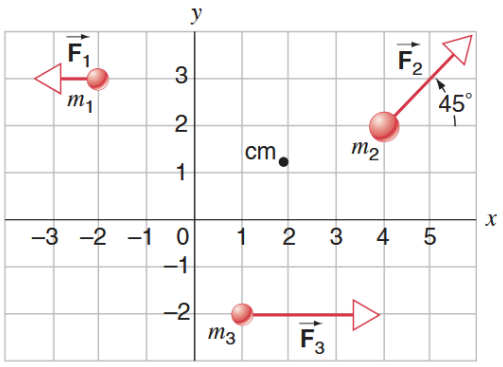
--> D
D = [2x2 double]
1.   0.
0.   3.

--> disp("Eigenvalues (diagonal entries
of D):"), disp(diag(D));
"Eigenvalues (diagonal entries of D):"
1.
3.

--> disp("Eigenvector matrix V (columns
are eigenvectors):"), disp(V);
"Eigenvector matrix V (columns
are eigenvectors):"
-0.7071068   0.7071068
0.7071068   0.7071068
```

EXERCISES

1. As shown in the figure below, a system of three particles of masses $m_1 = 4.1$ kg, $m_2 = 8.2$ kg and $m_3 = 4.1$ kg is initially at rest. The particles are each acted on by different net external forces, which have magnitudes $F_1 = 6$ N, $F_2 = 12$ N and $F_3 = 14$ N, whose directions are also shown in the figure. Write a Scilab program to compute the magnitude of the acceleration of the center of mass.



2. Given the matrix $\boldsymbol{A} = \begin{bmatrix} 4 & 1 & 2 \\ 0 & 3 & -1 \\ 1 & 0 & 2 \end{bmatrix}$

   a) Write the Scilab command to compute $|\boldsymbol{A}|$ and state whether $\boldsymbol{A}$ is invertible.

   b) If invertible, write the Scilab command to compute $\boldsymbol{A}^{-1}$.

   c) Write the Scilab command to compute the eigenvalues of $\boldsymbol{A}$.